

Saturation-Aware AutoQuantize: Pricing Mixed-Precision Configurations with Aumann–Shapley Attributions

Joshua Hill
Baseten Labs, Inc.

September 28, 2026

1 Introduction

Mixed-precision quantization keeps a model’s most sensitive layers at FP8 or BF16 and pushes the rest to NVFP4. The quality of the result depends almost entirely on how a layer’s sensitivity to quantization is measured. Current methods measure it in one of two ways. Some expand the loss with a low order Taylor expansion [1] and use Hessian-trace scores such as HAWQ-v2 [3]. Others quantize each layer on its own and record how much the loss moves. Both approaches score a layer with every other layer held at full precision, and then add the scores up to price a configuration.

That last step is where they break. Quantization damage *saturates*: the loss of a heavily quantized model is finite, and each additional quantized layer costs less than it would have cost on its own: the loss of quantizing two layers together is typically well below the sum of the losses of quantizing each alone.

This blog describes a saturation-aware auto-quantization method for the Model-Optimizer library [2]. It models the damage of quantizing any subset of layers with a closed-form *coverage* model, recovers that model’s parameters from a handful of gradient passes using Aumann–Shapley path integrals, and uses a linear program to find a quantization allocation [2]. The allocation it returns minimizes predicted damage under a memory budget, and it comes with an estimate of how much damage that allocation will cause.

2 Method

Throughout, let $U = \{1, \dots, L\}$ be the set of layers we consider quantizing (a single linear layer, or a group such as fused QKV that must share one format), and for $S \subseteq U$ let

$$f(S) = \mathbb{E}_x[\text{KL}(p(\cdot | x) \| p_S(\cdot | x))] \tag{1}$$

be the damage of quantizing the layers in S , measured as the KL divergence from the full-precision model’s next-token distribution. Notice that scoring requires no labels, and $f(\emptyset) = 0$. We describe a single target quantization format (e.g. NVFP4) first and return to multi-format (e.g. quantizing to NVFP4 and FP8) in Section 2.3.

2.1 The coverage model

Starting from some mild assumptions on how loss from quantizing layers should accumulate, we show that f must have a certain parametric form. Specifically, suppose damage accumulates one layer at a time according to a fixed rule,

$$f(S \cup \{i\}) = f(S) \oplus d_i, \quad i \notin S, \quad (2)$$

where d_i depends only on layer i , and the operation $\oplus$ is associative, continuous, strictly increasing, and has identity 0. These are mild requirements: the order in which layers are quantized should not matter, and quantizing a more harmful layer should cause more damage. The Aczél–Ling representation theorem [4, 5] states that there is a strictly increasing generator g with $x \oplus y = g(g^{-1}(x) + g^{-1}(y))$. It follows that

$$f(S) = g\left(\sum_{i \in S} b_i\right), \quad b_i = g^{-1}(d_i). \quad (3)$$

It follows from Equation (3). that the marginal cost of quantizing a layer depends on the curvature of g . Specifically

$$f(S \cup \{i\}) - f(S) = g\left(\sum_{j \in S} b_j + b_i\right) - g\left(\sum_{j \in S} b_j\right) = \int_{\sum_{j \in S} b_j}^{\sum_{j \in S} b_j + b_i} g'(u) du \quad (4)$$

And thus the saturating behaviour of quantizing a layer is a result of g' . Note that prior methods of summing sensitivities correspond to $g(x) = x$, and hence, no saturation.

Interactions between quantized layers are also captured by g . For instance,

$$f(\{i, j\}) - f(\{i\}) - f(\{j\}) = \int_0^{b_j} \int_0^{b_i} g''(s+t) dt ds \approx g''(0) b_j b_i \quad (5)$$

assuming g'' is close to constant over the domain of integration.

We use the *coverage* generator $g(u) = c(1 - e^{-u})$, which gives

$$f(S) = c\left(1 - \prod_{i \in S} (1 - a_i)\right), \quad a_i = 1 - e^{-b_i} \in [0, 1), \quad (6)$$

with a ceiling c and a per-layer break-rate a_i . The intuition is as follows: the loss is at most c , and each quantized layer consumes a fixed fraction a_i of whatever headroom remains, $c - f(S \cup \{i\}) = (1 - a_i)(c - f(S))$. A model that is already badly damaged has little headroom left, so quantizing one more layer adds little. The marginal cost of layer i in context S makes this explicit:

$$f(S \cup \{i\}) - f(S) = c a_i \prod_{j \in S} (1 - a_j), \quad (7)$$

which shrinks as S grows. The isolated sensitivity $f(\{i\}) = c a_i$ is the largest this marginal ever gets, and it overstates the in-context marginal by a factor of $\exp(\sum_{j \in S} b_j)$, an error that compounds when isolated scores are summed over many layers (Figure 1).

The coverage model works well in practice, and only has one parameter per layer and a ceiling parameter [2].

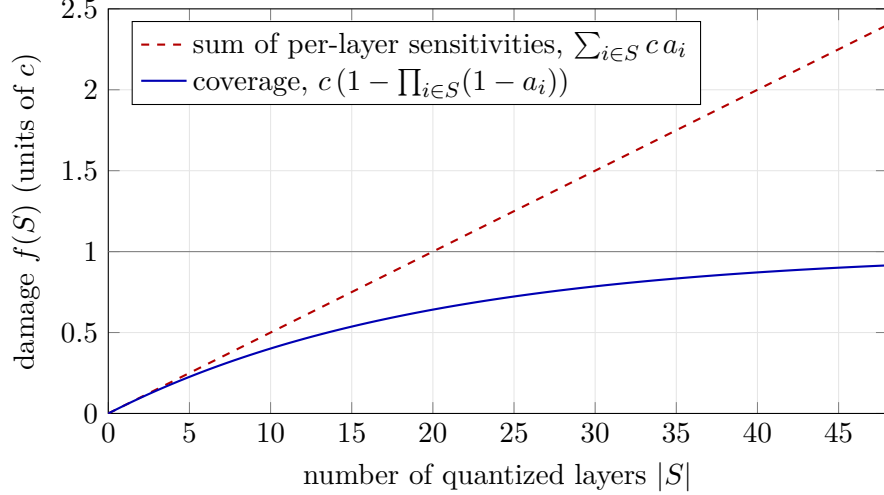


Figure 1: Illustration with $L = 48$ layers of equal break-rate $a_i = 0.05$. Both curves agree on every single layer, but the sum of per-layer sensitivities ignores the shrinking headroom and overprices the fully quantized model by roughly 160%. The gray line is the ceiling c .

2.2 Recovering the coverage parameters with Aumann–Shapley

Fitting $a_1, \dots, a_L$ and c by regression would require measuring the damage of many sampled configurations, one forward pass each. An alternative fitting procedure uses gradient information.

For each layer i with output y_i , let $\delta_i = Q(y_i) - y_i$ be its quantization error, and define the path

$$y_i(t) = y_i + t \delta_i \quad \text{for all } i \text{ simultaneously,} \quad t \in [0, 1], \quad (8)$$

so that $t = 0$ is the full-precision model and $t = 1$ the fully quantized one. Writing $F(t)$ for the KL divergence at point t , the Aumann–Shapley attribution [6] of layer i is the gradient projected onto its quantization error, averaged along the path:

$$\phi_i = \int_0^1 \langle \nabla_{y_i} F(t), \delta_i \rangle dt \approx \frac{1}{K} \sum_{k=0}^{K-1} \langle \nabla_{y_i} F(t_k), \delta_i \rangle, \quad t_k = \frac{k + \frac{1}{2}}{K}. \quad (9)$$

The attributions are *complete*: by the chain rule they sum exactly to the damage of the fully quantized model, $\sum_i \phi_i = F(1) - F(0) = f(U)$. Each one is measured with every other layer partly quantized, so it reflects the layer’s cost in context rather than in isolation. And one backward pass at each path node produces the attributions of all L layers at once. We use $K = 2$ nodes by default.

Under the coverage model, the attributions have a closed form. Integrating the gradient of Equation (6) along the diagonal gives

$$\phi_i = c a_i \int_0^1 \prod_{j \neq i} (1 - t a_j) dt, \quad (10)$$

Given the measured ϕ_i and the ceiling c , Equation (10) is a system of L equations in the L break-rates, which we solve by fixed-point iteration; the integrand is a polynomial in t , which Gauss–Legendre quadrature evaluates cheaply and to high accuracy.

The result is a break-rate a_i , and hence a cost b_i , for every layer, recovered from K backward passes and one forward pass. These costs are the inputs to the allocator in Section 2.3.

Model / method	effective bits				
	4.2	4.5	5.0	6.0	8.0
Qwen3-30B, AutoQuantize (gradient)	.0429	.0383	.0331	.0253	.0215
Qwen3-30B, saturation-aware	.0365	.0317	.0266	.0202	.0106
Qwen3-235B, AutoQuantize (gradient)	.0222	.0180	.0151	.0128	.0096
Qwen3-235B, saturation-aware	.0172	.0161	.0132	.0112	.0090

Table 1: Calibration KL divergence (lower is better) of the allocation at each effective-bits budget, formats {NVFP4, FP8, BF16}.

2.3 Solving for the configuration

With several candidate formats, we run one path per format φ (every layer moves toward φ at once) and obtain a cost $b_{i,\varphi}$ for each layer and format, with $b_{i,\text{BF16}} = 0$. The predicted damage of an assignment $\varphi(\cdot)$ is then

$$\hat{f} = c \left(1 - \exp \left(- \sum_i b_{i,\varphi(i)} \right) \right). \quad (11)$$

Since $\hat{f}$ is increasing in the summed cost, minimizing it under an effective-bits budget $\bar{b}$ is a linear program over one-hot choices $x_{i,\varphi} \in \{0, 1\}$:

$$\min_x \sum_{i,\varphi} b_{i,\varphi} x_{i,\varphi} \quad \text{s.t.} \quad \sum_{\varphi} x_{i,\varphi} = 1 \quad \forall i, \quad \sum_{i,\varphi} N_i \text{bits}(\varphi) x_{i,\varphi} \leq \bar{b} \sum_i N_i, \quad (12)$$

where N_i is the parameter count of layer i . The nonlinearity of saturation lives entirely in the mapping from $\sum b$ to damage. The same structure supports the reverse question: to find the smallest model whose predicted damage stays below a tolerance ε .

3 Results

Lower damage at matched memory. Table 1 compares calibration KL divergence at matched effective bits against the ModelOpt AutoQuantize gradient baseline, on two MoE models from 30B to 355B parameters. Factoring in saturation and layer-interactions results in a higher-quality quantization, with up to a 37% reduction in KL to the base model.

Quoting damage before deployment. Because the coverage model predicts damage rather than only ranking layers, every returned configuration carries a `predicted_damage` estimate.

3.1 Time and memory

The scoring cost does not depend on how many configurations the solver considers. Per calibration batch, the method runs one full-precision forward pass (the reference), one fully quantized forward pass (to determine c), and one forward-backward pass per quantization format (NVFP4 and FP8) and path node. With two quantization formats and $K = 2$ nodes that is four forward-backward and two forward passes. Like the gradient method, each scored layer replays its own forward with the candidate quantizers active, so the work scales as $O(N_{\text{layers}} \times N_{\text{formats}} \times K)$.

Scoring method	Passes per batch	Scoring time	Peak GPU memory
Gradient (Taylor)	1 fwd+bwd	~16 min	29 GB
Saturation-aware	$N_{\text{formats}}K$ fwd+bwd + 2 fwd	~40 min	29 GB
KL divergence	one full model per layer and format	~14 h	23 GB

Table 2: Scoring cost on Qwen3.6-35B-A3B, 4× RTX 6000 Ada, 128 samples at sequence length 512. Gradient and KL rows from [1].

3.2 Usage

The method is selected through the existing `method` argument. Because the loss is KL divergence against the model’s own outputs, it needs a `forward_step` that returns logits, and no labels or `loss_func`.

```
import modelopt.torch.quantization as mtq

model, search_state = mtq.auto_quantize(
    model,
    constraints={"effective_bits": 4.8},
    quantization_formats=[mtq.NVFP4_DEFAULT_CFG, mtq.FP8_DEFAULT_CFG],
    data_loader=calib_loader,
    forward_step=lambda model, batch: model(**batch).logits,
    method={"method": "aumann_shapley", "num_path_nodes": 2},
    num_calib_steps=512,
    num_score_steps=128,
)
print(search_state["best"]["predicted_damage"]) # mean per-token KL
```

To search for the smallest configuration within a damage tolerance instead, drop the effective-bits constraint and pass `{"method": "aumann_shapley", "max_predicted_damage": 1e-2}`.

4 Future work

A better proxy for inference cost. Effective bits measures memory, and memory is not speed. Realized speedup depends on the serving stack: which formats have kernels, how layers are fused, and how often each weight is read. For an MoE model decoding at small batch, a routed expert’s bytes are read far less often than an attention projection’s, so two configurations with equal effective bits can differ substantially in decode throughput. Replacing effective bits with measured per-layer latency, or with weight bytes read per decoded token, would let the solver optimize the quantity users actually care about. The linear program accepts any additive cost, so this is a change of inputs, not of method.

5 Conclusion

Existing mixed-precision methods score each layer as if it were the only one quantized, then add the scores. Quantization damage saturates, so that sum overprices the aggressive configurations a tight budget requires. We model the damage of any subset of layers with a closed-form coverage model, in which each layer consumes a fraction of the remaining headroom, and recover its parameters from a few gradient passes along the path to the fully quantized model. The costs it produces are

additive, so the allocation remains solvable with a linear program, and the result is a configuration with lower damage at the same memory, together with an estimate of what that damage will be.

References

- [1] Model Optimizer Team. AutoQuantize: A fast automatic mixed-precision assignment. <https://nvidia.github.io/Model-Optimizer/announcements/autoquantize.html>, 2026.
- [2] J. Hill. Saturation makes quantization error additive: A coverage model with a certificate. arXiv:2607.12266, 2026.
- [3] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, and K. Keutzer. HAWQ-V2: Hessian aware trace-weighted quantization of neural networks. *NeurIPS*, 2020.
- [4] J. Aczél. *Lectures on Functional Equations and Their Applications*. Academic Press, 1966.
- [5] C.-H. Ling. Representation of associative functions. *Publicationes Mathematicae Debrecen*, 12:189–212, 1965.
- [6] R. J. Aumann and L. S. Shapley. *Values of Non-Atomic Games*. Princeton University Press, 1974.